

From Automated Program Repair to Automated Agent Repair: Why Failed Trajectories Matter

Xiangxin Zhao

University College London
London, United Kingdom
xiangxin.zhao.21@ucl.ac.uk

Abstract

Automated Program Repair (APR) has shown that buggy programs can often be repaired rather than rewritten. As LLM coding agents increasingly perform software engineering tasks, an analogous question arises: *can failed agent executions be repaired instead of restarted?* We answer the prerequisite question with, to the best of our knowledge, the first execution-grounded audit of failed coding-agent trajectories. Our preliminary results show that failed trajectories retain substantial recoverable value, completing a median of 75% of task subgoals ($n = 27$), while none of the audited agents ever states the correct fix (0/26). These findings establish the empirical foundation for *Trajectory-Centric Automated Agent Repair (TC-AAR)*, extending APR from programs to agent executions.

CCS Concepts

• **Software and its engineering** → *Software verification and validation*.

Keywords

automated program repair, coding agents, failed trajectories, agent repair

ACM Reference Format:

Xiangxin Zhao. 2026. From Automated Program Repair to Automated Agent Repair: Why Failed Trajectories Matter. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3832783.3844579>

1 Introduction

Automated Program Repair (APR) has substantially reduced the cost of debugging by automatically repairing buggy programs [9], from GenProg [8], Prophet [11], and SPR [10] to recent LLM-based approaches [2, 7, 17, 22, 24, 26]. As LLM coding agents [14, 19–21] increasingly perform software engineering tasks autonomously, an analogous question naturally arises: *can failed agent executions be repaired instead of restarted?*

Current practice simply retries from scratch whenever an agent fails. This is wasteful: restarting recomputes work the failed execution may have already completed correctly, and discards the

reasoning and environment knowledge accumulated during execution. Whether failed trajectories actually contain reusable value, however, remains unknown.

Our prior work [27] showed that coding agents typically commit to failure early: the decisive error occurs at a median of Step 7 in a median 27-step trajectory. This suggests that much of the execution occurs after the fatal mistake. However, that study focused on *how* failures happen, not *what* they leave behind. To the best of our knowledge, no prior work has measured how much recoverable value survives in failed trajectories.

This paper takes a first step toward *Automated Agent Repair (AAR)* by answering this fundamental question with an execution-grounded audit that decomposes task grading into step-level executable checkpoints and replays failed trajectories in their original environments.

Our contributions are (i) to the best of our knowledge, the first execution-grounded methodology for measuring recoverable value in failed coding-agent trajectories, and (ii) an empirical foundation for AAR and a research agenda.

2 Related Work

Automated Program Repair: Two decades of APR show that buggy programs can be repaired, not rewritten, with tests as the oracle [2, 8–11, 23, 26]. The plastic surgery hypothesis [1] sharpened its premise: repair material usually already exists in the codebase. We lift this question to trajectories: does the material for repair already exist inside a *failed agent execution*?

Coding Agents: Coding agents [14, 19–21] interleave reasoning and execution over long horizons, so one early mistake shapes every later step [27]. Yet recovery today discards this history: retry restarts from scratch, and Reflexion-style self-reflection [16] critiques the failing model inside the very context that misled it. Neither treats the failed execution as evidence to be salvaged (cf. Evidence 3).

Failure Trajectory Analysis: Prior studies characterize agent behavior [3, 12, 27], attribute failures to steps or agents [6, 18, 25], build failure taxonomies [4, 15], question outcome-only benchmarks [28], or distill terminal-task benchmarks from in-the-wild recordings [5]. All of these explain *why* runs fail; none weighs *how much correct work* a failed run leaves behind: precisely what a repair system would harvest. Our audit fills this gap: it is post hoc and execution-grounded, judging progress by replaying the run rather than by trusting the agent’s claims.

3 Motivation and Feasibility of AAR

Motivation. Before developing repair algorithms, AAR must answer a more fundamental question: **are failed trajectories worth repairing?** Binary pass/fail outcomes cannot quantify how much



This work is licensed under a Creative Commons Attribution 4.0 International License. ASE '26, Munich, Germany

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
<https://doi.org/10.1145/3832783.3844579>

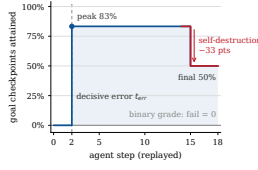


Figure 1: A failed trajectory may contain substantial recoverable value: this run reaches 83% completion, then destroys part of its own progress and is graded a failure.

Table 1: Empirical evidence for AAR feasibility (“—” = not applicable).

Finding	Metric	Value	n/N
Retained progress	median final attainment (%), n=27)	75	—
Majority progress	runs attaining $\geq 50\%$	—	26/27
Mis-graded successes	fully correct, graded failed	—	2/27
Self-destruction	median peak-progress loss (%)	33	4/27
Environment facts	mean verified facts/run (n=26)	2.1	—
Self-diagnosis	runs stating the correct fix	—	0/26
Grader blind spots	requirements no test covers (18 tasks)	—	20

reusable progress survives failure, so we conduct an execution-grounded feasibility audit (intuition in Figure 1).

Experimental setup. Our audit covers failed trajectories of seven LLMs under three scaffolds on terminal-bench [13]; we replay the two with clean command logs (mini-swe-agent [21], OpenHands [19]), as Terminus 2 records raw screen output. We refine each task’s end-state grading into executable *checks* from four sources: *Goal* checks translate the official tests (103 across 18 tasks); *Enabler* checks come from the reference solution; *Constraint* checks cover instruction requirements no test verifies; *Hazard* checks flag destructive actions. An LLM drafts every check; we review each manually. We replay each failed trajectory command by command in its original Docker environment, running all checks after every step. For fidelity, we first replay each task’s *successful* runs and drop tasks below 80%. From the 1,184 annotated failures of [27] we audit a stratified sample of 41; headline numbers use the strictest 27 (tasks replaying at 100%); reasoning-side findings use 26.

Table 1 summarizes the evidence. *Evidence 1: Failed trajectories preserve substantial reusable progress.* Failed trajectories attain a median of 75% of task goal checks (n=27), and 26/27 (96%) complete at least half, showing that restarting discards substantial completed work.

Evidence 2: Recoverable execution state survives failure. 4/27 trajectories later destroyed a median of 33% of their peak progress: repair should resume from the best execution state rather than the final failed state.

Evidence 3: External repair is necessary. Despite substantial progress and 2.1 verified environment facts per run on average, no audited agent ever states the correct fix (0/26): external repair is necessary.

4 Research Questions

Building AAR raises three key research questions:

- **RQ1 (Localization):** Can the decisive error be localized automatically?

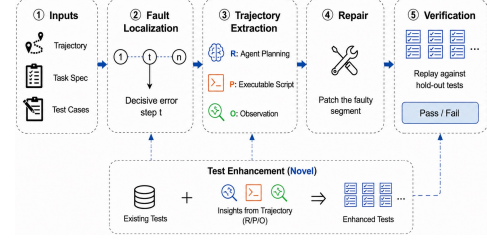


Figure 2: TC-AAR: localize the fault, distill reusable knowledge (R/P/O), repair the trajectory, and enhance verification.

- **RQ2 (Repair):** How should the faulty trajectory be repaired?
- **RQ3 (Verification):** How can repaired trajectories be verified without leaking the solution?

5 Approach

Figure 2 presents Trajectory-Centric Automated Agent Repair (TC-AAR), which extends APR from program code to execution trajectories, treating failed trajectories as reusable assets that preserve progress, environment knowledge, and reasoning.

Fault localization. TC-AAR first identifies the decisive error t_{err} : the earliest step after which the trajectory irreversibly commits to failure. Localization lets repair focus on the faulty segment while preserving all validated execution before t_{err} . **Novelty:** fault localization over execution trajectories rather than program code.

Knowledge extraction. TC-AAR extracts three representations of reusable knowledge: **Reasoning (R)**, the agent’s planning process; **Programs (P)**, executable scripts and commands; and **Observations (O)**, verified information obtained from the environment. **Novelty:** failed trajectories become reusable knowledge assets, not discarded logs.

Repair and resumption. TC-AAR patches only the faulty segment: a local edit (replace/delete/insert) of the actions at and after t_{err} by the original or a dedicated repair agent, then *resumed* from the best preserved state. **Novelty:** trajectory editing replaces full regeneration, analogous to patch generation in APR.

Validation and test enhancement. Repaired runs are re-validated by all checkpoints and the official grader, and compared against restart baselines on success, tokens, tool calls, and environment steps. Trajectory knowledge further augments test suites with execution-informed assertions, targeting the 20 coverage gaps our audit exposed. **Novelty:** failed trajectories also strengthen benchmark verification. A falsifiable hypothesis follows: *TC-AAR can match or exceed the task success of full restart at lower token, tool-call, and interaction cost.*

6 Conclusion

This work establishes an empirical foundation for Automated Agent Repair (AAR): our execution-grounded audit shows that failed coding-agent trajectories preserve substantial reusable progress, yet agents themselves are rarely able to recover it. These findings motivate *Trajectory-Centric Automated Agent Repair (TC-AAR)*, which treats failed trajectories as reusable computational assets rather than discarded logs.

References

- [1] Earl T. Barr, Yuriy Brun, Premkumar Devanbu, Mark Harman, and Federica Sarro. 2014. The Plastic Surgery Hypothesis. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. Association for Computing Machinery, 306–317. doi:10.1145/2635868.2635898
- [2] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2188–2200. doi:10.1109/ICSE55347.2025.00157
- [3] Islem Bouzenia and Michael Pradel. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE. arXiv:2506.18824 [cs.SE] doi:10.48550/arXiv.2506.18824
- [4] Mert Cemri, Melissa Z. Pan, Shuyi Yang, et al. 2025. Why Do Multi-Agent LLM Systems Fail?. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. arXiv:2503.13657 [cs.AI] doi:10.48550/arXiv.2503.13657
- [5] Zhaoyang Chu, Jiarui Hu, Xingyu Jiang, Pengyu Zou, Han Li, Chao Peng, Peter O'Hearn, Earl T. Barr, Mark Harman, Federica Sarro, and He Ye. 2026. TerminalWorld: Benchmarking Agents on Real-World Terminal Tasks. arXiv:2605.22535 [cs.SE] doi:10.48550/arXiv.2605.22535
- [6] Yu Ge, Linna Xie, Zhong Li, Yu Pei, and Tian Zhang. 2025. Who is Introducing the Failure? Automatically Attributing Failures of Multi-Agent Systems via Spectrum Analysis. arXiv:2509.13782 [cs.SE] doi:10.48550/arXiv.2509.13782
- [7] Dong Huang, Jie M. Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. 2023. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. arXiv:2312.13010 [cs.CL] doi:10.48550/arXiv.2312.13010
- [8] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. GenProg: A Generic Method for Automatic Software Repair. *IEEE Transactions on Software Engineering* 38, 1 (2012), 54–72. doi:10.1109/TSE.2011.104
- [9] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. 2019. Automated Program Repair. *Commun. ACM* 62, 12 (Dec. 2019), 56–65. doi:10.1145/3318162
- [10] Fan Long and Martin Rinard. 2015. Staged Program Repair with Condition Synthesis. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. Association for Computing Machinery, 166–178. doi:10.1145/2786805.2786811
- [11] Fan Long and Martin Rinard. 2016. Automatic Patch Generation by Learning Correct Code. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. Association for Computing Machinery, 298–312. doi:10.1145/2837614.2837617
- [12] Oorja Majgaonkar, Zhiwei Fei, Xiang Li, Federica Sarro, and He Ye. 2025. Understanding Code Agent Behaviour: An Empirical Study of Success and Failure Trajectories. arXiv:2511.00197 [cs.SE] doi:10.48550/arXiv.2511.00197
- [13] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, et al. 2026. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. In *Proceedings of the International Conference on Learning Representations (ICLR)*. arXiv:2601.11868 [cs.AI] doi:10.48550/arXiv.2601.11868
- [14] Yue Pan, Zimin Chen, Siyu Lu, et al. 2025. Prometheus: Towards Long-Horizon Codebase Navigation for Repository-Level Problem Solving. arXiv:2507.19942 [cs.SE] doi:10.48550/arXiv.2507.19942
- [15] Priyam Sahoo, Gaurav Mittal, Xiaomin Li, Shengjie Ma, Benjamin Steenhoeck, Pingping Lin, and Yu Hu. 2026. AgentLens: Revealing The Lucky Pass Problem in SWE-Agent Evaluation. arXiv:2605.12925 [cs.SE] doi:10.48550/arXiv.2605.12925
- [16] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R. Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*. <https://openreview.net/forum?id=vAEIhFcKW6>
- [17] Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. 2024. MAGIS: LLM-Based Multi-Agent Framework for GitHub Issue Resolution. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2403.17927 [cs.SE] doi:10.48550/arXiv.2403.17927
- [18] Minxing Wang, Xiaofei Xie, and Yintong Huo. 2026. TrajAudit: Automated Failure Diagnosis for Agentic Coding Systems. arXiv:2605.26563 [cs.SE] doi:10.48550/arXiv.2605.26563
- [19] Xingyao Wang, Boxuan Li, Yufan Song, et al. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *Proceedings of the International Conference on Learning Representations (ICLR)*. doi:10.48550/arXiv.2407.16741
- [20] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE, Article FSE037 (2025), 24 pages. doi:10.1145/3715754
- [21] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2405.15793 [cs.SE] doi:10.48550/arXiv.2405.15793
- [22] He Ye, Matias Martinez, Xiapu Luo, Tao Zhang, and Martin Monperrus. 2022. SelfAPR: Self-supervised Program Repair with Test Execution Diagnostics. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE) (ASE '22)*. Association for Computing Machinery, New York, NY, USA, Article 92, 13 pages. doi:10.1145/3551349.3556926
- [23] He Ye, Matias Martinez, and Martin Monperrus. 2022. Neural Program Repair with Execution-based Backpropagation. In *Proceedings of the 44th International Conference on Software Engineering (ICSE) (ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 1506–1518. doi:10.1145/3510003.3510222
- [24] He Ye and Martin Monperrus. 2024. ITER: Iterative Neural Repair for Multi-Location Patches. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE) (Lisbon, Portugal) (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 10, 13 pages. doi:10.1145/3597503.3623337
- [25] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 76583–76599. <https://proceedings.mlr.press/v267/zhang25cq.html>
- [26] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. Association for Computing Machinery, 1592–1604. doi:10.1145/3650212.3680384
- [27] Xiangxin Zhao, Han Li, Shuaiting Li, Tianyi Zhao, Earl T. Barr, Federica Sarro, and He Ye. 2026. Failure as a Process: An Anatomy of CLI Coding Agent Trajectories. arXiv:2607.09510 [cs.SE] doi:10.48550/arXiv.2607.09510 Under review.
- [28] Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy K. Zhang, Shu Liu, et al. 2025. Establishing Best Practices in Building Rigorous Agentic Benchmarks. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. arXiv:2507.02825 [cs.AI] doi:10.48550/arXiv.2507.02825